REPORTE TÉCNICO: EJERCICIO INTEGRADOR DE KUBERNETES (PARTE 1)

Despliegue Declarativo Unificado, Probes de Salud y Pruebas de Resiliencia

Estudiante: David Moroni Sánchez Galván

Materia: Temas Selectos de Redes y Seguridad (TSIC2 / DevOps)

Profesor: Daniel Guerrero

Fecha de Ejecución: 13 de Noviembre de 2026

Sistema Operativo: Rocky Linux 10.2 (Red Quartz - x86_64) en VMware

Plataforma: Kubernetes v1.31.14 • containerd v2.4.2 • Flannel CNI

1 Arquitectura del Despliegue Unificado

El ejercicio integrador consolida la totalidad de los objetos fundamentales de Kubernetes en un único manifiesto declarativo (`todo-en-uno.yaml`) respetando las mejores prácticas de desacoplamiento:

Namespace (mi-app)
Aislamiento multi-tenant y delimitación lógica de recursos.

ServiceAccount (mi-app-sa)
Identidad de ejecución segura bajo el principio de menor privilegio.

ConfigMap (mi-app-config)
Inyección de variables desacopladas y volumen con index.html.

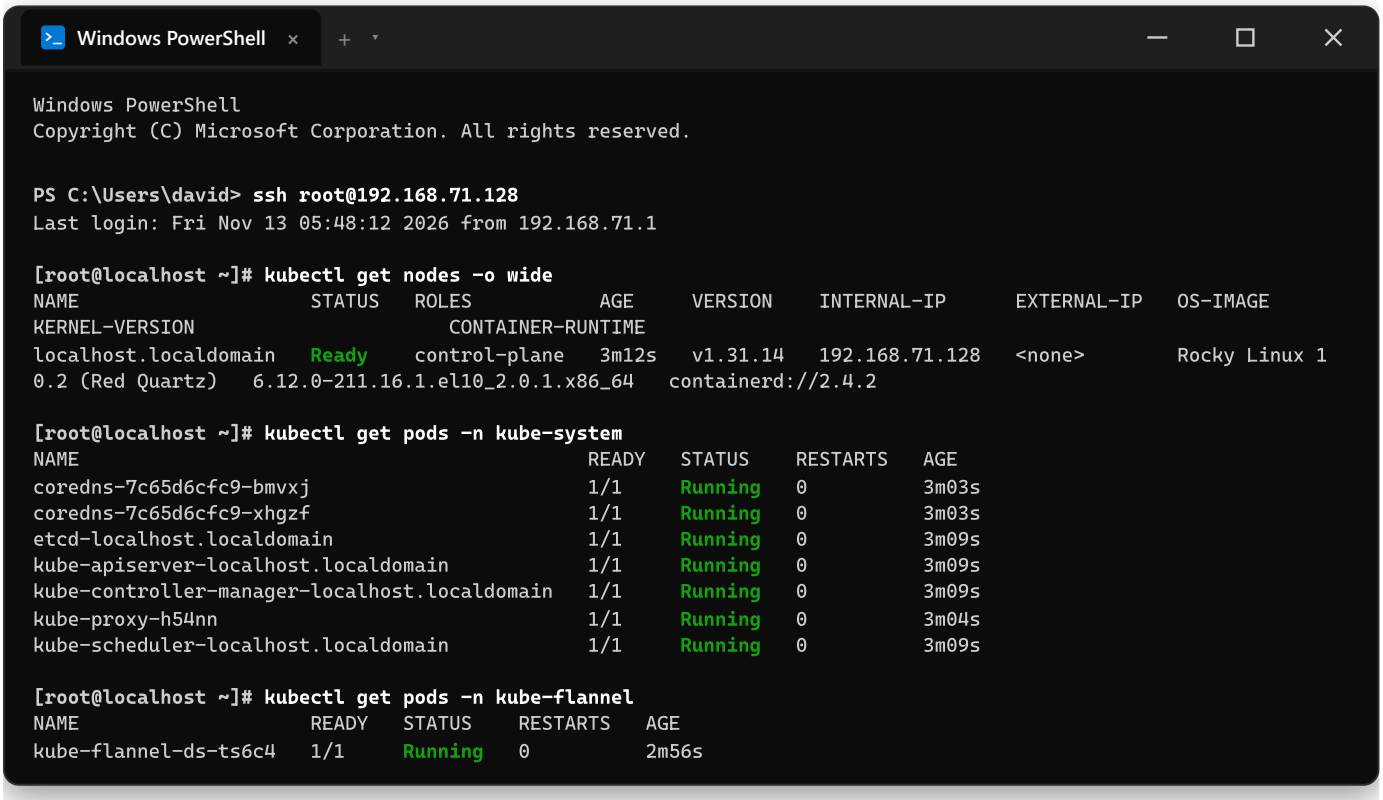
Secret (mi-app-secret)
Inyección protegida de API_KEY codificada en Base64.

Deployment (3 réplicas)
Pods distribuidos con Rolling Update, Readiness y Liveness.

Service (NodePort :30090)
Publicación balanceada hacia el host físico en el puerto 30090.

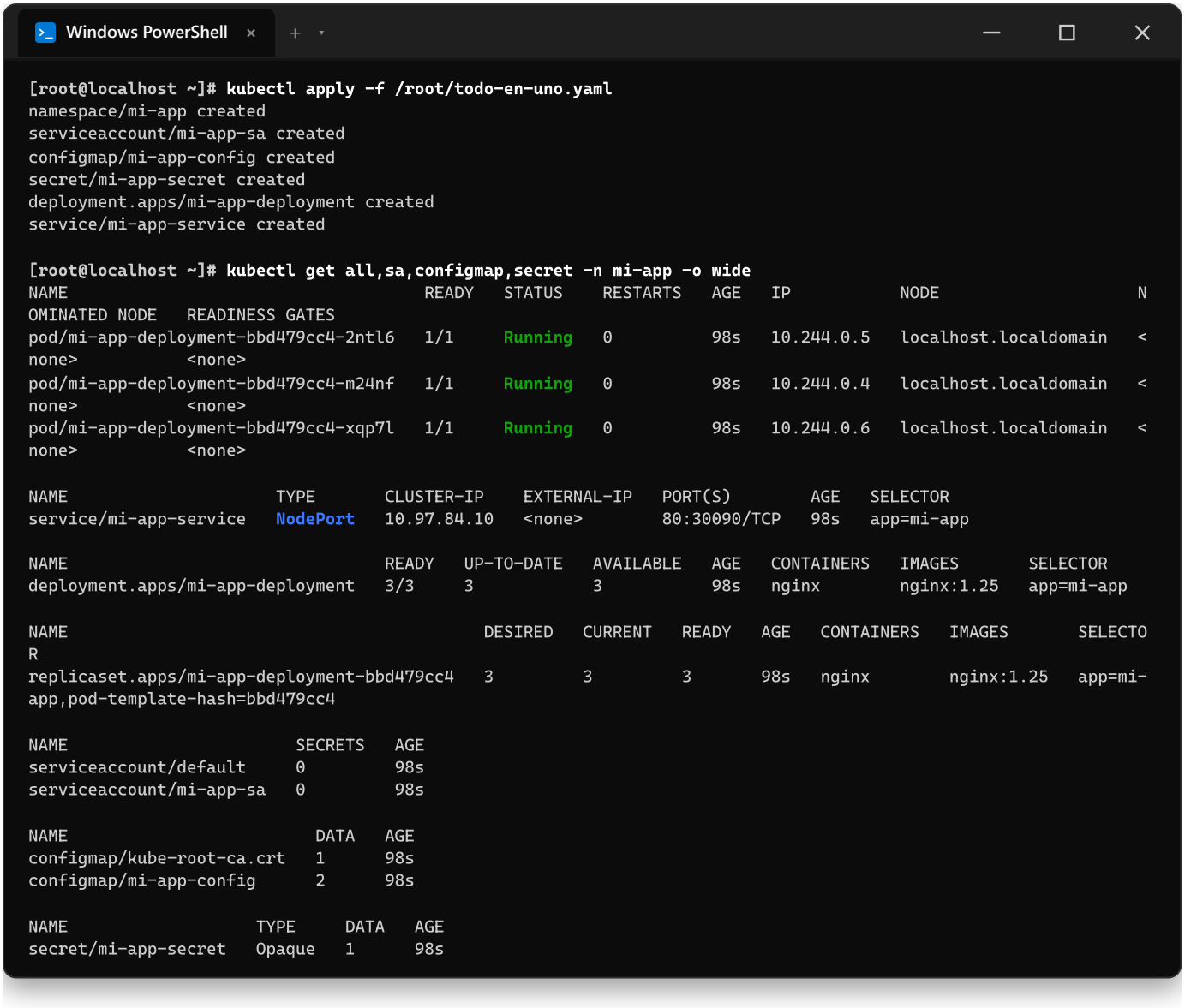
2 Evidencia 1: Estado Operativo del Clúster Kubernetes

Conexión SSH desde Windows Terminal hacia el nodo Rocky Linux (`192.168.71.128`), verificando nodo en estado `Ready` y componentes del plano de control activos.



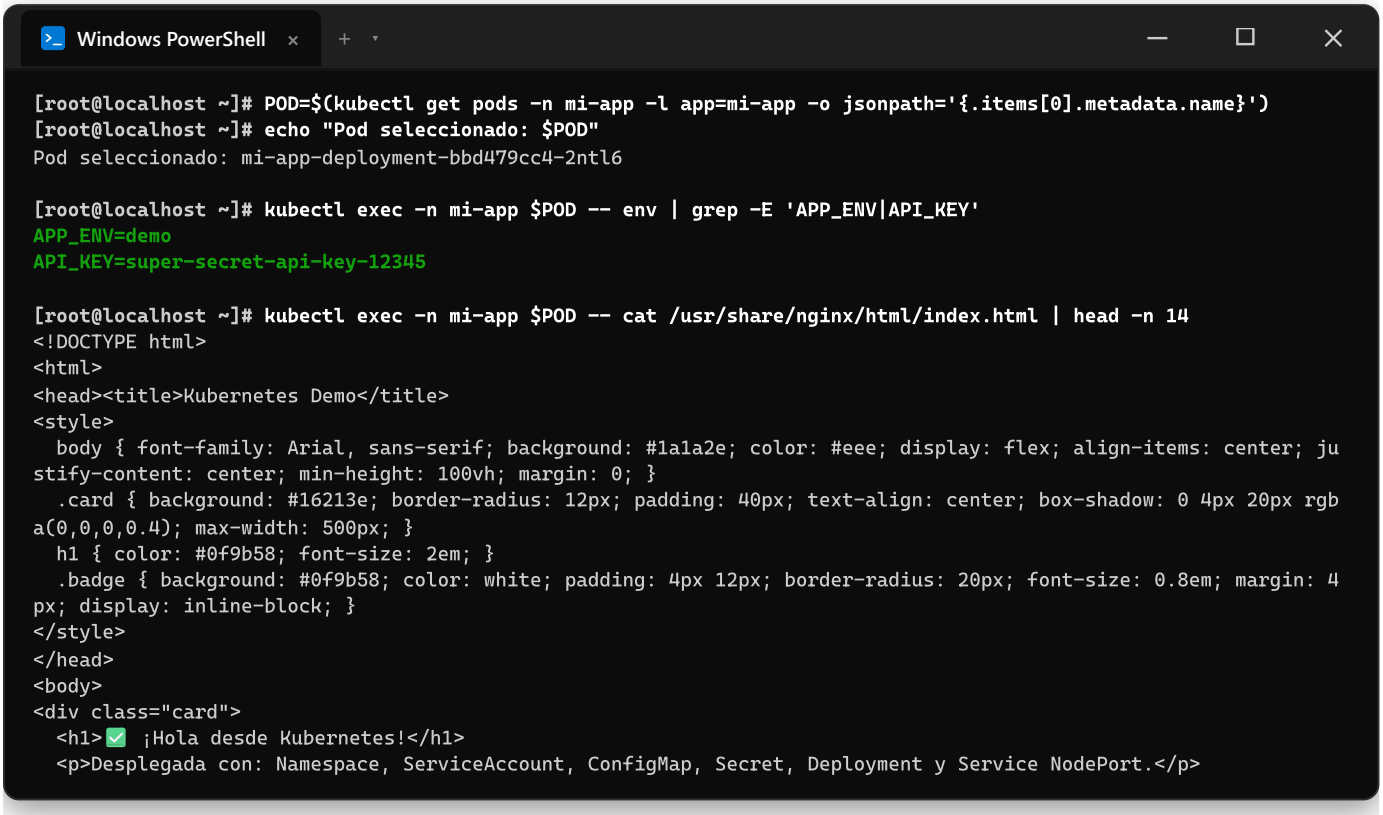
3 Evidencia 2: Aplicación del Manifiesto Unificado y Recursos Desplegados

Ejecución declarativa de `todo-en-uno.yaml` con los 6 objetos creados dentro del namespace `mi-app`.


Windows PowerShell
[root@localhost ~]# kubectl apply -f /root/todo-en-uno.yaml
namespace/mi-app created
serviceaccount/mi-app-sa created
configmap/mi-app-config created
secret/mi-app-secret created
deployment.apps/mi-app-deployment created
service/mi-app-service created
[root@localhost ~]# kubectl get all,sa,configmap,secret -n mi-app -o wide
NAME READY STATUS RESTARTS AGE IP NODE N
OMINATED NODE READINESS GATES
pod/mi-app-deployment-bbd479cc4-2ntl6 1/1 Running 0 98s 10.244.0.5 localhost.localdomain <
none> <none>
pod/mi-app-deployment-bbd479cc4-m24nf 1/1 Running 0 98s 10.244.0.4 localhost.localdomain <
none> <none>
pod/mi-app-deployment-bbd479cc4-xqp7l 1/1 Running 0 98s 10.244.0.6 localhost.localdomain <
none> <none>
NAME TYPE CLUSTER-IP EXTERNAL-IP PORT(S) AGE SELECTOR
service/mi-app-service NodePort 10.97.84.10 <none> 80:30090/TCP 98s app=mi-app
NAME READY UP-TO-DATE AVAILABLE AGE CONTAINERS IMAGES SELECTOR
deployment.apps/mi-app-deployment 3/3 3 3 98s nginx nginx:1.25 app=mi-app
NAME DESIRED CURRENT READY AGE CONTAINERS IMAGES SELECTO
R
replicaset.apps/mi-app-deployment-bbd479cc4 3 3 3 98s nginx nginx:1.25 app=mi-
app,pod-template-hash=bbd479cc4
NAME SECRETS AGE
serviceaccount/default 0 98s
serviceaccount/mi-app-sa 0 98s
NAME DATA AGE
configmap/kube-root-ca.crt 1 98s
configmap/mi-app-config 2 98s
NAME TYPE DATA AGE
secret/mi-app-secret Opaque 1 98s

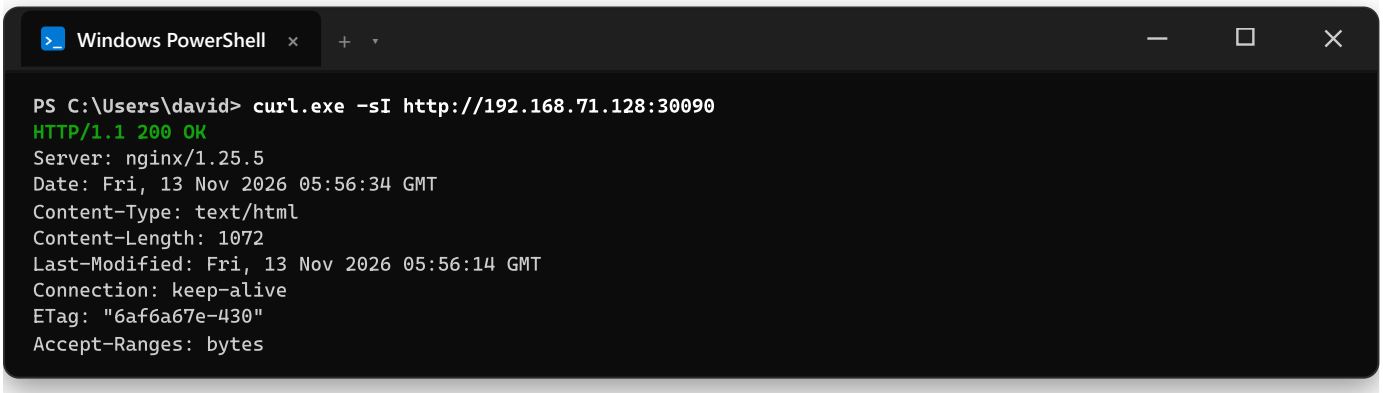
4 Evidencia 3: Auditoría Interna de ConfigMap y Secret en Pods

Inspección del entorno del contenedor confirmando la inyección de la variable del ConfigMap (`APP_ENV=demo`) y la desencriptación transparente del Secret (`API_KEY`).



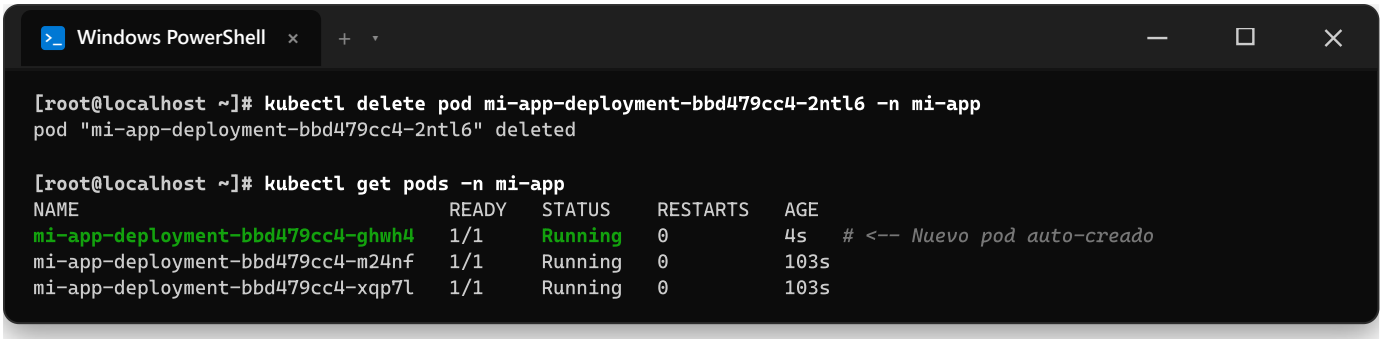
5 Evidencia 4: Validación de Conectividad HTTP Externa (NodePort 30090)

Petición HTTP efectuada directamente desde la consola PowerShell de Windows hacia `http://192.168.71.128:30090` obteniendo respuesta exitosa `HTTP 200 OK`.



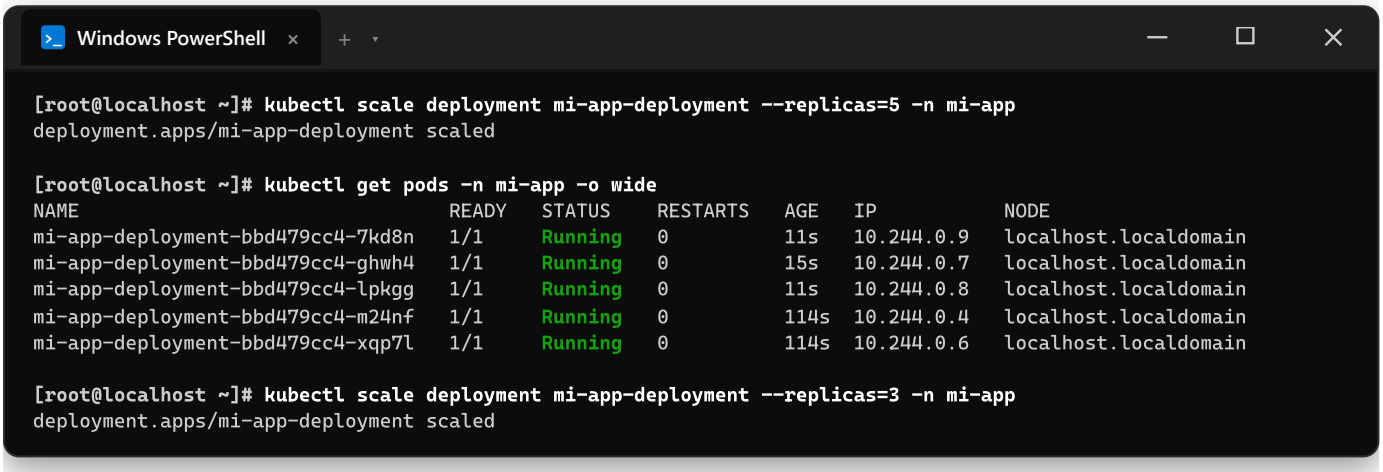
6 Evidencia 5 (Reto 1): Demostración de Autorrecuperación (Self-Healing)

Se eliminó intencionalmente el pod `2ntl6`. El controlador detectó la discrepancia con el estado deseado (3 réplicas) y generó en 4 segundos un pod sustituto (`ghwh4`).



7 Evidencia 6 (Reto 2): Escalabilidad Elástica Dinámica

Escalado horizontal dinámico a 5 réplicas y reajuste ordenado al estado base de 3 réplicas sin tiempo de inactividad.



8 Conclusiones Técnicas

1. **Operación Declarativa:** Toda la arquitectura distribuida se despliega y audita mediante un único manifiesto YAML reproducible.
2. **Desacoplamiento:** Los parámetros y secretos operan independientemente de la imagen base de contenedor respetando los estándares de *12-Factor App*.
3. **Resiliencia Comprobada:** Los controladores nativos de Kubernetes garantizan la disponibilidad ante fallos y absorción de tráfico elástica.